

ПЛАН ВЫПОЛНЕНИЯ: Beatleap for Windows

Инструкция для Claude Code Opus

ОБЯЗАТЕЛЬНЫЕ ПРАВИЛА (читать перед каждым шагом)

1. **Всегда держи оба файла открытыми:** `CLAUDE_EXECUTION_PLAN.md` (этот файл) и `Beatleap_Windows_Desktop_TZ.md` (ТЗ). Перед каждым шагом перечитывай указанные разделы ТЗ.
 2. **Не упрощай.** Если в ТЗ указан точный HEX-цвет, размер, алгоритм или FFmpeg-команда — используй именно их, без замены на «похожие».
 3. **Не добавляй функционал**, которого нет в ТЗ (никаких «бонусных» фич, экранов подписки, авторизации, водяных знаков).
 4. **Один шаг — одна задача.** Не переходи к следующему шагу, пока текущий не завершён и не проверен.
 5. **После каждого файла** показывай его полное содержимое, не обрезай.
 6. **Если что-то неясно** — смотри в ТЗ, не придумывай. Если в ТЗ нет ответа — спроси пользователя.
 7. **Подписка, paywall, авторизация** — не реализовывать ни в каком виде.
-

СТАТУС ВЫПОЛНЕНИЯ

Отмечай каждый завершённый шаг: `[]` → `[x]`

- ☐ Шаг 1: Инициализация проекта
- ☐ Шаг 2: Дизайн-система, роутинг, State Management
- ☐ Шаг 3: Экраны MediaPicker и MusicPicker
- ☐ Шаг 4: Beat Detection (Python + IPC)
- ☐ Шаг 5: Алгоритм нарезки и генерации монтажа
- ☐ Шаг 6: Главный экран редактора (Layout)
- ☐ Шаг 7: Вкладка TOOLS (все 7 инструментов)

- ☐ Шаг 8: Вкладка EDIT (9 эффектов)
 - ☐ Шаг 9: Вкладка FILTERS + FFmpeg Pipeline
 - ☐ Шаг 10: Экспорт, горячие клавиши, обработка ошибок
 - ☐ Шаг 11: Финальная проверка по Acceptance Criteria
-

ШАГ 1 — Инициализация проекта

Разделы ТЗ: § 2 (Технологический стек), § 3 (Структура файлов)

Задача:

Создай полную структуру проекта Electron.js + React 18 + TypeScript + Tailwind CSS точно по дереву файлов из раздела 3 ТЗ. Настрой все конфигурационные файлы так, чтобы проект запускался и открывал пустое тёмное окно (фон `#0D0D0D`).

Что создать:

- `package.json` со всеми зависимостями: `electron` , `react` , `react-dom` , `typescript` , `tailwindcss` , `zustand` , `fluent-ffmpeg` , `electron-builder`
- `tsconfig.json` с настройками для Electron + React
- `tailwind.config.js`
- `electron/main.ts` — главный процесс, открывает окно 1280×800, минимум 1024×600
- `electron/preload.ts` — preload-скрипт с IPC bridge (`contextBridge`)
- Все пустые папки и файлы-заглушки согласно дереву из § 3 ТЗ
- `src/App.tsx` — корневой компонент с роутером (пока одна пустая страница)

Проверка шага: Команда `npm run dev` запускает окно Electron с тёмным фоном без ошибок в консоли.

ШАГ 2 — Дизайн-система, роутинг, State Management

Разделы ТЗ: § 4 (Дизайн-система), § 5 (список экранов), § 15 (State Management)

Задача:

Реализуй три независимых блока.

Блок А — Дизайн-система:

В `tailwind.config.js` и глобальном CSS (`src/index.css`) задай все CSS-переменные из таблицы § 4 ТЗ с точными HEX-значениями. Подключи шрифт Inter через Google Fonts

или локально. Создай базовые Tailwind-классы для кнопок (primary, secondary), карточек и панелей.

Блок В — Роутинг:

В `src/App.tsx` реализуй роутинг между 5 экранами: `HomeScreen`, `MediaPickerScreen`, `MusicPickerScreen`, `ProcessingScreen`, `EditorScreen`. Каждый экран — отдельный компонент в `src/screens/`. Пока каждый экран содержит только заголовок с названием на тёмном фоне. Переходы между экранами — через Zustand (поле `currentScreen`), не через React Router.

Блок С — Zustand Store:

В `src/store/projectStore.ts` реализуй полный TypeScript-интерфейс `ProjectState` точно по коду из § 15 ТЗ. Все поля обязательны. Добавь экшены: `setCurrentScreen`, `setMediaFiles`, `setSelectedTrack`, `setMood`, `setDuration`, `setFormat`, `setFade`, `setActiveEffect`, `setActiveFilter`, `setFilterIntensity`, `setBeatData`, `setGeneratedClips`, `setIsProcessing`, `setIsExporting`, `setExportProgress`.

Проверка шага: Переключение `currentScreen` в store меняет отображаемый экран. TypeScript не выдаёт ошибок.

ШАГ 3 — Экраны выбора медиа и музыки

Разделы ТЗ: § 5.1 (HomeScreen), § 5.2 (MediaPickerScreen), § 5.3 (MusicPickerScreen)

Задача:

Реализуй три экрана полностью, включая UI и логику.

HomeScreen (§ 5.1):

Логотип «Beatleap» (Inter SemiBold, 36px, `--accent-green`), подзаголовок, кнопка «Начать» (240×56px, фон `--accent-green`, текст чёрный, скругление 28px). Нажатие переводит на MediaPickerScreen.

MediaPickerScreen (§ 5.2):

- Верхняя панель: кнопка «× Отмена», заголовок «ВЫБОР ВИДЕО», кнопка «Далее →» (неактивна пока нет файлов)
- Основная область: сетка 4 колонки с миниатюрами видео. Поддержка выбора файлов через системный диалог (фильтр: .mp4, .mov, .avi) и через Drag-and-Drop в окно
- При выборе файла — зелёная галочка поверх миниатюры, длительность в правом нижнем углу

- Нижняя лента (высота 120px): выбранные клипы 80×80px с порядковым номером и кнопкой «X». Перетаскивание для изменения порядка. Кнопки «Перемешать» и «По порядку»
- Кнопка «Далее» активна при ≥ 1 файле, переводит на MusicPickerScreen

MusicPickerScreen (§ 5.3):

- Верхняя панель: «← Назад», «ВЫБОР МУЗЫКИ», «Далее →»
- Горизонтальные табы категорий: FEATURED, POP, RAP, CINEMATIC, SPORT, TRAVEL, ACOUSTIC, FUNK, HOUSE. Активная — цвет `--accent-green` с подчёркиванием
- Список треков: строка 72px, обложка 48×48px, название + исполнитель, длительность. Клик — воспроизведение превью. Выбранный трек — синяя точка справа
- Нижние вкладки: BEATLEAP (встроенная библиотека), ФАЙЛЫ (системный диалог .mp3/.wav/.aac)
- Встроенная библиотека: создай JSON-файл `src/data/tracks.json` с минимум 5 треками-заглушками (название, исполнитель, категория, длительность, путь к файлу). Реальные MP3-файлы добавить в `assets/music/`
- Кнопка «Далее» переводит на ProcessingScreen

Проверка шага: Можно выбрать видеофайлы, изменить их порядок, выбрать трек и перейти к обработке.

ШАГ 4 — Beat Detection (Python + IPC)

Разделы ТЗ: § 9.1 (Python-скрипт), § 2 (стек), § 14 (fallback при ошибке Python)

Задача:

Реализуй полный модуль анализа аудио.

Python-скрипт `python/beat_detect.py` :

Реализуй точно по коду из § 9.1 ТЗ. Скрипт принимает путь к аудиофайлу как аргумент командной строки и выводит в stdout JSON-объект с полями: `tempo` (float), `beat_times` (array of float), `onset_times` (array of float), `duration` (float). Обработка исключений: при любой ошибке выводить `{"error": "описание ошибки"}`.

IPC-обработчик `electron/ipc/audio.ts` :

Реализуй IPC-канал `analyze-audio`. Принимает путь к аудиофайлу. Запускает `python/beat_detect.py` через `child_process.spawn`. Парсит stdout как JSON. Возвращает результат в renderer-процесс. Таймаут: 30 секунд.

Утилита `src/utls/beatDetection.ts` :

Экспортирует функцию `analyzeBeat(audioPath: string): Promise<BeatData>` . Вызывает IPC-канал `analyze-audio` . При ошибке (Python недоступен или скрипт упал) — активирует fallback из § 14 T3: равномерное разбиение трека на фрагменты по 0.5 секунды.

Preload обновление:

Добавь в `electron/preload.ts` экспозицию метода `window.electronAPI.analyzeAudio(path)` .

Проверка шага: Вызов `analyzeBeat` с реальным MP3-файлом возвращает корректный объект `BeatData` с массивами временных меток.

ШАГ 5 — Алгоритм нарезки и генерации монтажа

Разделы T3: § 9.2 (точки склейки), § 9.3 (распределение фрагментов), § 9.4 (расстановка эффектов), § 5.4 (ProcessingScreen)

Задача:

Реализуй алгоритм генерации монтажа и экран обработки.

`src/utls/videoSlicer.ts` :

Экспортирует функцию `generateClips(beatData, mediaFiles, mood, duration, segmentStart, mediaOrder)` . Логика:

- Mood `mellow` : каждый 4-й элемент `beat_times`
- Mood `natural` : каждый 2-й элемент `beat_times`
- Mood `energetic` : все `beat_times` + каждый 3-й `onset_times` , объединить и отсортировать
- Обрезать массив точек склейки до `segmentStart + duration`
- Сдвинуть все метки на `segmentStart`
- Распределить фрагменты из `mediaFiles` по точкам: длина фрагмента = разница между соседними точками
- Не использовать один и тот же клип более 2 раз подряд
- Возвращает массив `GeneratedClip[]` : `{ sourceFile, startTime, duration, effectSlots: [] }`

`src/utls/effectsEngine.ts` :

Экспортирует функцию `applyEffects(clips, activeEffects, beatTimes)` . Логика:

- Для каждого активного эффекта (уровень 1 или 2) выбрать случайные метки из `beatTimes`
- Уровень 1: 20% меток, уровень 2: 40% меток

- Метки одного эффекта не ближе 0.5 сек друг к другу
- Записать эффект в `effectSlots` ближайшего клипа
- Возвращает обновлённый массив `GeneratedClip[]`

ProcessingScreen (§ 5.4):

- Полноэкранный тёмный фон
- Анимированный пульсирующий круг (CSS-анимация `pulse` , цвет `--accent-green`)
- Текст «Синхронизируем видео с ритмом...»
- Линейный прогресс-бар 320px
- Текст текущего шага: «Анализируем аудио...» → «Нарезаем клипы...» → «Применяем эффекты...» → «Готово!»
- Последовательность вызовов: `analyzeBeat()` → `generateClips()` → `applyEffects()` → переход на EditorScreen
- По завершении сохранить результат в store (`setGeneratedClips` , `setBeatData`)

Проверка шага: После выбора медиа и музыки экран обработки проходит все этапы и переходит в редактор с заполненным store.

ШАГ 6 — Главный экран редактора (Layout)

Разделы ТЗ: § 5.5 (EditorScreen макет), компоненты Timeline, VideoPreview

Задача:

Реализуй полный макет EditorScreen с тремя зонами.

Зона А — Верхняя панель (высота 52px, фон `--bg-secondary` , `border-bottom` `border`):

- Слева: иконка «Домой» (при клике — диалог подтверждения «Вернуться на главную? Прогресс будет потерян.»)
- По центру: логотип «Beatleap» (Inter SemiBold, 20px, `--accent-green`)
- Справа: кнопка «Сохранить» (120×36px, фон `--accent-green` , текст чёрный, скругление 18px)

Зона В — Центральная область (60% ширины):

- `VideoPreview` компонент: HTML5 `<video>` элемент, сохраняет соотношение сторон из store (`format`). По умолчанию 9:16. Фон `#000000` . Отображает первый кадр первого клипа как превью

- Под плеером: кнопка Play/Pause (круглая 40px, фон `--bg-tertiary`), иконка ноты (36px, переход к MusicPickerScreen), иконка Shuffle (36px, повторная генерация)
- `Timeline` компонент (высота 24px): горизонтальная полоса, ползунок-scrubber (белый круг 16px), перетаскивание мышью
- Над таймлайном: полоса маркеров эффектов (высота 8px) — оранжевые точки диаметром 6px в позициях из `beatData.beat_times`, на которых есть активные эффекты

Зона С — Правая панель (40% ширины, фон `--bg-secondary`):

- Три вкладки TOOLS / EDIT / FILTERS (высота 44px). Активная: белый текст + подчёркивание `--accent-green`. Неактивная: `--text-secondary`
- Содержимое вкладок — пустые компоненты-заглушки `ToolsPanel`, `EditPanel`, `FiltersPanel` (заполнятся в шагах 7–9)

Проверка шага: EditorScreen корректно отображается с тремя зонами, переключение вкладок работает, scrubber перетаскивается.

ШАГ 7 — Вкладка TOOLS (все 7 инструментов)

Разделы ТЗ: § 6 (все подразделы 6.1–6.7)

Задача:

Реализуй компонент `ToolsPanel` с полным функционалом всех 7 инструментов.

Интерфейс панели: Горизонтальная прокручиваемая лента иконок с подписями (иконка 40px + подпись 11px). При клике на иконку — соответствующий контент разворачивается в нижней части правой панели или открывается модальное окно.

Videos (§ 6.1): Модальное окно с функционалом MediaPickerScreen. Кнопки «Отмена» и «Применить». При «Применить» — обновить `mediaFiles` в store и запустить повторную генерацию (вернуться на ProcessingScreen).

Tweak (§ 6.2): Модальное окно `TweakModal` (80% ширины экрана):

- Верхняя часть: горизонтальная лента миниатюр всех `generatedClips` (80×60px, временная метка под каждым)
- При клике на фрагмент — выделить его и показать кнопки «Highlight» и «Replace»
- **Highlight:** filmstrip-вид исходного видео (горизонтальная полоса кадров). Текущий диапазон подсвечен. Перетаскивание окна выбора. Кнопка «Применить» обновляет `tweakOverrides` в store
- **Replace:** системный диалог выбора файла. Заменяет фрагмент в `generatedClips`

Duration (§ 6.3): Вертикальный список пресетов из таблицы § 6.3 T3 (Snap 0:10, Story 0:15, Reels 0:30, TikTok 1:00, Facebook 1:30, Popular 3:00, Полный трек). Клик обновляет `duration` в store и запускает пересчёт.

Segment (§ 6.4): Визуализация waveform аудиотрека (Canvas или SVG). Прозрачное окно выделения шириной, пропорциональной `duration`. Перетаскивание окна обновляет `segmentStart` в store.

Mood (§ 6.5): Три карточки Mellow / Natural / Energetic с описанием. Клик обновляет `mood` в store и запускает пересчёт.

Fade (§ 6.6): Сетка 2×2 из карточек: None, In, Out, All. Клик обновляет `fade` в store.

Format (§ 6.7): Три карточки: Portrait 9:16, Square 1:1, Landscape 16:9. Клик обновляет `format` в store и меняет размер плеера в VideoPreview.

Проверка шага: Все 7 инструментов открываются, изменения сохраняются в store, пересчёт запускается там, где требуется.

ШАГ 8 — Вкладка EDIT (9 эффектов)

Разделы T3: § 7 (полный список эффектов и их реализация)

Задача:

Реализуй компонент `EditPanel` с полным функционалом.

Интерфейс: Сетка круглых иконок-переключателей (диаметр 64px). Под каждой иконкой — название. Состояния:

- Уровень 0 (выкл): фон `--bg-tertiary`, иконка серая, без точек
- Уровень 1: фон `--bg-tertiary`, обводка `--accent-green` 2px, 1 оранжевая точка под иконкой
- Уровень 2: обводка `--accent-green` 2px, 2 оранжевые точки под иконкой
- Клик циклически меняет уровень: 0 → 1 → 2 → 0

9 эффектов (иконки, названия и FFmpeg-реализация — строго по таблице § 7 T3): Fast Cut, Flash, Zoom, Prism, RGB, Boomerang, Split, Hue, Speed.

Кнопка Shuffle: Иконка двух скрещенных стрелок над сеткой. При нажатии вызывает `applyEffects()` с новым случайным seed, обновляет `generatedClips` в store, обновляет маркеры на Timeline.

Обновление маркеров: При изменении любого эффекта — пересчитать позиции оранжевых точек на Timeline.

Проверка шага: Все 9 эффектов переключаются, маркеры на таймлайне обновляются.

ШАГ 9 — Вкладка FILTERS + FFmpeg Pipeline

Разделы ТЗ: § 8 (фильтры), § 10 (FFmpeg Pipeline)

Задача:

Реализуй фильтры и полный движок рендеринга.

FiltersPanel компонент (§ 8):

- Горизонтальная лента карточек фильтров (72×100px). Первая карточка — «Нет»
- Каждая карточка: миниатюра первого кадра видео с применённым фильтром (генерировать через FFmpeg при загрузке) + название
- Выбранный фильтр: рамка `--accent-green` 2px
- Ползунок «Интенсивность» (0–100) под лентой
- Клик обновляет `activeFilter` и `filterIntensity` в store

10 фильтров (строго по таблице § 8 ТЗ): Нет, Warm, Cool, Vintage, B&W, VCR, Glitch, Film, Light Leak, Vignette.

src/utils/ffmpegBuilder.ts (§ 10):

Экспортирует функцию `buildAndRender(project: ProjectState, outputPath: string, quality: '720p'|'1080p'|'4k', onProgress: (percent: number) => void): Promise<void>` .

Реализуй 6-этапный pipeline строго по § 10 ТЗ:

1. Нарезка каждого фрагмента: `ffmpeg -ss [start] -t [duration] -i [input] [fragment_N.mp4]`
2. Применение эффектов к каждому фрагменту (FFmpeg-фильтры из таблицы § 7)
3. Применение глобального фильтра (FFmpeg-команды из таблицы § 8) с учётом `filterIntensity`
4. Склейка через `concat` фильтр
5. Наложение аудио, обрезка до нужной длины, применение `fade` если выбрано
6. Финальный экспорт: `ffmpeg -i [concat] -c:v libx264 -preset fast -crf 23 -c:a aac -b:a 192k [output.mp4]`

Разрешения: 720p = 1280×720, 1080p = 1920×1080, 4K = 3840×2160. Соотношение сторон берётся из `format` в store.

Прогресс: парсить вывод FFmpeg (`frame=` , `time=`) для расчёта `percent` и вызова `onProgress` .

Проверка шага: Рендеринг тестового проекта (2–3 клипа, 15 сек) завершается успешно, файл MP4 воспроизводится.

ШАГ 10 — Экспорт, горячие клавиши, обработка ошибок

Разделы ТЗ: § 11 (ExportModal), § 13 (горячие клавиши), § 14 (обработка ошибок)

Задача:

Реализуй финальные элементы приложения.

ExportModal компонент (§ 11):

- Открывается при нажатии кнопки «Сохранить» в верхней панели
- Заголовок «Экспорт видео»
- Выпадающий список «Качество»: 720p (HD), 1080p (Full HD), 4K (Ultra HD). По умолчанию 1080p
- Кнопка «Выбрать папку» — системный диалог выбора директории. Показывает выбранный путь
- Кнопка «Экспортировать» (фон `--accent-green`)
- При нажатии «Экспортировать»: закрыть модальное окно, показать нижнюю панель прогресса в EditorScreen
- Нижняя панель прогресса: прогресс-бар + текст «Рендеринг... X%» + кнопка «Отмена»
- Кнопка «Отмена»: прерывает FFmpeg-процесс (`process.kill()`), удаляет временные файлы, скрывает панель прогресса
- По завершении: toast-уведомление «Видео сохранено!» с кнопкой «Открыть папку» (открывает директорию в проводнике Windows через `shell.openPath`)

Горячие клавиши (§ 13):

Реализовать через `useEffect` с `window.addEventListener('keydown')` в корневом компоненте:

Клавиша	Действие
<code>Space</code>	Play / Pause в VideoPreview
<code>Ctrl+Z</code>	Отмена последнего действия (undo через store)
<code>Ctrl+S</code>	Открыть ExportModal

Ctrl+N	Новый проект (диалог подтверждения → HomeScreen)
Ctrl+O	Открыть диалог добавления медиафайлов
Ctrl+R	Перемешать монтаж (повторный вызов generateClips)
1	Переключить на вкладку TOOLS
2	Переключить на вкладку EDIT
3	Переключить на вкладку FILTERS

Обработка ошибок (§ 14):

Реализовать все 5 edge cases строго с текстами из § 14 ТЗ:

1. Неподдерживаемый формат — toast с текстом из § 14
2. Ошибка FFmpeg — диалоговое окно с текстом из § 14
3. Слишком короткое видео (<5 сек суммарно) — предупреждение при нажатии «Далее» в MediaPickerScreen
4. Python недоступен — fallback-режим (равномерная нарезка по 0.5 сек)
5. Отмена экспорта — прерывание процесса и очистка временных файлов

Проверка шага: Все горячие клавиши работают. Все 5 ошибочных сценариев воспроизводятся и обрабатываются корректно.

ШАГ 11 — Финальная проверка по Acceptance Criteria

Разделы ТЗ: § 16 (Acceptance Criteria)

Задача:

Пройди по каждому пункту § 16 ТЗ и убедись, что все критерии выполнены. Для каждого пункта напиши результат проверки.

Чеклист функциональных требований:

- ☐ Выбор от 1 до 30 видеофайлов работает корректно
- ☐ Выбор трека из встроенной библиотеки работает
- ☐ Загрузка собственного аудиофайла работает
- ☐ Автоматическая генерация монтажа завершается без ошибок

- ☐ Все 9 эффектов вкладки EDIT видны в предпросмотре
- ☐ Все 10 фильтров вкладки FILTERS применяются корректно
- ☐ Все 7 инструментов вкладки TOOLS функционируют
- ☐ Экспорт в 720p завершается успешно
- ☐ Экспорт в 1080p завершается успешно
- ☐ Экспорт в 4K завершается успешно
- ☐ Все горячие клавиши работают на всех экранах

Чеклист требований к производительности:

- ☐ Анализ аудио 3 мин $\leq$ 10 сек
- ☐ Генерация монтажа из 10 клипов $\leq$ 30 сек
- ☐ Экспорт 60 сек в 1080p $\leq$ 2 мин

Чеклист требований к UI/UX:

- ☐ Все цвета соответствуют палитре § 4 ТЗ
- ☐ Переходы между экранами плавные (200–300 мс)
- ☐ Корректная работа при разрешении 1280×720
- ☐ Корректная работа при разрешении 1920×1080
- ☐ Корректная работа при разрешении 2560×1440

Если какой-либо пункт не выполнен — исправить до отметки [x] .

ФИНАЛЬНАЯ КОМАНДА ДЛЯ СБОРКИ

После прохождения всех шагов выполни:

Bash

```
npm run build
```

Результат: установочный файл .exe для Windows в папке dist/ . Проверь, что приложение запускается на чистой Windows-машине без установки дополнительного ПО.

Этот файл является частью проекта Beatleap for Windows. Основное ТЗ: [Beatleap_Windows_Desktop_TZ.md](#)